Lecture 28.11: Post-Pruning Implementation — Minimal Cost-Complexity Pruning

📌 Executive Summary

Minimal Cost-Complexity Pruning mathematics (Breiman CART), effective alpha sequence extraction, cost-complexity path traversal, and finding optimal ccp_alpha via cross-validation.

🏛️ Theoretical Formulation & Mathematical Architecture

📐 Minimal Cost-Complexity Pruning Mathematics

Formulated by Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone (1984), **Minimal Cost-Complexity Pruning** defines a parameterized objective function balancing empirical misclassification error with structural tree complexity:

$$\boxed{R_\alpha(T) = R(T) + \alpha|T|}$$

Where:

- $R(T) = \sum_{t \in \mathcal{L}} R(t) = \sum_{t \in \mathcal{L}} \left(\frac{N_t}{N} I(t)\right)$ is the total unpenalized leaf impurity/error.
- $|T|$ is the number of terminal leaf nodes in subtree T.
- $\alpha \geq 0$ is the **complexity parameter** (regularization penalty).

Pruning Mechanics:

For each internal node t:

1. If node t is collapsed into a single leaf, its error is $R(t)$.
2. If subtree T_t rooted at t is kept, its error is $R(T_t) + \alpha|T_t|$.
3. Subtree T_t and leaf t have identical cost-complexity when:

$$R(t) + \alpha(1) = R(T_t) + \alpha|T_t| \implies \alpha = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

This critical value is the **effective alpha** for node t. Pruning the node with the smallest effective alpha produces the optimal nested subtree sequence:

$$T_0 \supset T_1 \supset T_2 \supset \cdots \supset \{t_{\text{root}}\}$$

💻 Vectorized Python Implementation

```
from sklearn.tree import DecisionTreeClassifier

# Compute Cost-Complexity Pruning Path
base_clf = DecisionTreeClassifier(random_state=42).fit(X_syn, y_syn)
path = base_clf.cost_complexity_pruning_path(X_syn, y_syn)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

print(f"Total Effective Alphas: {len(ccp_alphas)}")
print("First 3 Alphas: ", np.round(ccp_alphas[:3], 5))
print("Final 3 Alphas: ", np.round(ccp_alphas[-3:], 5))

# Fit with mid-range alpha
pruned_clf = DecisionTreeClassifier(random_state=42, ccp_alpha=ccp_alphas[len(ccp_
pruned_clf.fit(X_syn, y_syn)
print(f"Unpruned Nodes: {base_clf.tree_.node_count} -> Pruned Nodes: {pruned_clf.t
assert pruned_clf.tree_.node_count < base_clf.tree_.node_count
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.